

3장 평가(Evaluation) — 정확도부터 ROC-AUC까지

목차

1. 3.1 정확도(Accuracy)

1. 정확도의 정의
2. MyDummyClassifier로 확인하는 정확도의 함정
3. 불균형 데이터에서의 정확도 — MNIST 실습

2. 3.2 오차 행렬(Confusion Matrix)

1. 오차 행렬의 구조
2. MyFakeClassifier의 오차 행렬

3. 3.3 정밀도(Precision)와 재현율(Recall)

1. 정의와 업무적 의미
2. MyFakeClassifier의 정밀도·재현율
3. 평가 함수 생성
4. 타이타닉 로지스틱 회귀 평가
5. 정밀도/재현율 트레이드오프
6. `predict_proba()` 이해
7. Binarizer 활용
8. 임계값 변화에 따른 지표 변화
9. `precision_recall_curve()`
10. 정밀도-재현율 곡선 시각화

4. 3.4 F1 스코어

5. 3.5 ROC 곡선과 AUC

1. ROC 곡선의 개념
2. `roc_curve()` 활용
3. ROC 곡선 시각화
4. `roc_auc_score()`
5. 최종 통합 평가 함수

6. 핵심 요약 및 머신러닝 활용 포인트

1. 3.1 정확도(Accuracy)

1.1 정확도의 정의

정확도(Accuracy)는 실제 데이터에서 예측 데이터가 얼마나 같은지를 판단하는 지표입니다.

정확도(Accuracy) = 예측 결과가 동일한 데이터 건수 / 전체 예측 데이터 건수

직관적으로 모델 예측 성능을 나타내는 평가 지표입니다. 하지만 **이진 분류의 경우 데이터의 구성에 따라 ML 모델의 성능을 왜곡할 수 있기 때문에** 정확도 수치 하나만 가지고 성능을 평가하지 않습니다.

1.2 MyDummyClassifier로 확인하는 정확도의 함정

```
import numpy as np
from sklearn.base import BaseEstimator

class MyDummyClassifier(BaseEstimator):
    # fit( ) 메소드는 아무것도 학습하지 않음.
    def fit(self, X, y=None):
        pass

    # predict( ) 메소드는 단순히 Sex feature가 1 이면 0 , 그렇지 않으면 1 로 예측함.
    def predict(self, X):
        pred = np.zeros( ( X.shape[0], 1 ))
        for i in range (X.shape[0]) :
            if X['Sex'].iloc[i] == 1:
                pred[i] = 0
            else :
                pred[i] = 1

        return pred
```

줄	코드	상세 설명
2	from sklearn.base import BaseEstimator	<code>BaseEstimator</code> 는 사이킷런의 모든 <code>Estimator</code> 가 상속하는 기반 클래스입니다. 이를 상속하면 <code>get_params()</code> , <code>set_params()</code> 등이 자동으로 제공되어 사이킷런 생태계와 호환되는 커스텀 모델을 만들 수 있습니다.
4	class MyDummyClassifier(BaseEstimator):	학습을 전혀 하지 않는 가짜 분류기 를 정의합니다. 정확도가 얼마나 쉽게 부풀려지는지 보여 주는 것이 목적입니다.
6~7	def fit(self, X, y=None): pass	<code>pass</code> 는 아무것도 하지 않는다 는 뜻입니다. 학습 과정이 완전히 비어 있습니다.
10	def predict(self, X):	예측 메서드입니다. 사이킷런 규약을 지키기 위해 이름을 <code>predict</code> 로 맞췄습니다.
11	pred = np.zeros((X.shape[0], 1))	입력 샘플 수만큼 0으로 채워진 배열 을 준비합니다.
12~16	if X['Sex'].iloc[i] == 1: pred[i] = 0 else: pred[i] = 1	단 하나의 규칙만 적용합니다. 2.6장에서 Sex는 female→0, male→1 로 인코딩되었으므로, 이 코드는 "남자면 사망(0), 여자면 생존(1)" 으로 예측하는 것입니다. 머신러닝이 아니라 사람이 만든 if-else 규칙 일 뿐입니다.

```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score

# 원본 데이터를 재로딩, 데이터 가공, 학습 데이터/테스트 데이터 분할.
titanic_df = pd.read_csv('./titanic_train.csv')
y_titanic_df = titanic_df['Survived']
X_titanic_df = titanic_df.drop('Survived', axis=1)
X_titanic_df = transform_features(X_titanic_df)
X_train, X_test, y_train, y_test = train_test_split(X_titanic_df, y_titanic_df,
                                                    test_size=0.2, random_state=0)

# 위에서 생성한 Dummy Classifier를 이용해 학습/예측/평가 수행.
myclf = MyDummyClassifier()
myclf.fit(X_train, y_train)

mypredictions = myclf.predict(X_test)
print('Dummy Classifier의 정확도는: {0:.4f}'.format(accuracy_score(y_test, mypredictions)))
```

줄	코드	상세 설명
6~9	titanic_df = pd.read_csv(...) / transform_features(...)	2.6장에서 만든 전처리 함수 (fillna, drop_features, format_features)를 그대로 재사용합니다. 결측치 처리, 불필요 컬럼 삭제, 레이블 인코딩이 한 번에 수행됩니다.
10~11	train_test_split(..., test_size=0.2, random_state=0)	학습 712건 / 테스트 179건으로 분리합니다.
14~15	myclf.fit(X_train, y_train)	호출은 하지만 내부가 pass라 아무 일도 일어나지 않습니다 .
17~18	accuracy_score(y_test, mypredictions)	결과는 0.7877 . 학습을 전혀 하지 않은 모델이 78.77%의 정확도 를 기록했습니다.

▶ 실행 결과

Dummy Classifier의 정확도는: 0.7877

⚠ 결과 해석 — 정확도의 첫 번째 함정

단순히 "여자는 살고 남자는 죽는다"는 규칙만으로 78.77%가 나왔습니다. 2.6장에서 실제로 학습시킨 **DecisionTreeClassifier의 정확도가 0.7877**이었던 것과 정확히 같습니다.

즉 **정확도만 보면 학습한 모델과 아무것도 안 한 모델을 구분할 수 없습니다**. 이는 타이타닉 데이터에서 성별이 워낙 강력한 변수이기 때문이며, 정확도가 **모델의 실력을 제대로 반영하지 못하는** 대표적인 사례입니다.

1.3 불균형 데이터에서의 정확도 — MNIST 실습

정확도의 더 심각한 문제는 **불균형한(imbalanced) 레이블 데이터**에서 드러납니다.

```
from sklearn.datasets import load_digits
from sklearn.model_selection import train_test_split
from sklearn.base import BaseEstimator
from sklearn.metrics import accuracy_score
import numpy as np
import pandas as pd

class MyFakeClassifier(BaseEstimator):
```

```
def fit(self,X,y):
    pass

# 입력값으로 들어오는 X 데이터 셋의 크기만큼 모두 0값으로 만들어서 반환
def predict(self,X):
    return np.zeros( (len(X), 1) , dtype=bool)

# 사이킷런의 내장 데이터 셋인 load_digits( )를 이용하여 MNIST 데이터 로딩
digits = load_digits()

print(digits.data)
print("### digits.data.shape:", digits.data.shape)
print(digits.target)
print("### digits.target.shape:", digits.target.shape)
```

줄	코드	상세 설명
8~14	class MyFakeClassifier(BaseEstimator):	앞의 Dummy보다 더 극단적입니다. 입력이 무엇이든 무조건 0(False)만 반환 합니다. 규칙조차 없습니다.
14	return np.zeros((len(X), 1), dtype=bool)	샘플 수만큼 False로 채운 배열 을 반환합니다. <code>dtype=bool</code> 이라 True/False로 표현됩니다.
17	digits = load_digits()	사이킷런 내장 손글씨 숫자 데이터 입니다. 8×8 픽셀 이미지를 64개 피쳐 로 펼친 형태이며 총 1797건입니다.
19~22	print(digits.data / target ...)	<code>data</code> 는 (1797, 64) 2차원, <code>target</code> 은 (1797,) 1차원입니다. <code>target</code> 값은 0~9의 숫자 레이블입니다.

▶ 실행 결과

```
[[ 0.  0.  5. ...  0.  0.  0.]
 [ 0.  0.  0. ... 10.  0.  0.]
 [ 0.  0.  0. ... 16.  9.  0.]
 ...
 [ 0.  0.  1. ...  6.  0.  0.]
 [ 0.  0.  2. ... 12.  0.  0.]
 [ 0.  0. 10. ... 12.  1.  0.]]
### digits.data.shape: (1797, 64)
[0 1 2 ... 8 9 8]
### digits.target.shape: (1797,)
```

```
digits.target == 7
```

줄	코드	상세 설명
1	digits.target == 7	NumPy의 불린 인덱싱 (1.3장)입니다. 비교 연산이 원소 단위로 일괄 적용 되어 1797개의 True/False 배열이 반환됩니다.

▶ 실행 결과

```
array([False, False, False, ..., False, False, False])
```

```
# digits번호가 7이면 True이고 이를 astype(int)로 1로 변환, 7번이 아니면 False이고 0으로 변환.
y = (digits.target == 7).astype(int)
X_train, X_test, y_train, y_test = train_test_split( digits.data, y, random_state=11)
```

줄	코드	상세 설명
---	----	-------

2	<code>y = (digits.target == 7).astype(int)</code>	10개 클래스의 다중 분류 문제를 이진 분류로 변환합니다. <code>astype(int)</code> 로 True→1, False→0이 됩니다. 결과적으로 "이 숫자가 7인가?"를 묻는 문제가 되며, 7은 전체의 약 10%뿐이라 자연스럽게 불균형 데이터가 만들어집니다.
3	<code>train_test_split(digits.data, y, random_state=11)</code>	<code>test_size</code> 를 지정하지 않아 기본값 0.25가 적용됩니다. $1797 \times 0.25 \approx 450$ 건이 테스트 세트가 됩니다.

```
# 불균형한 레이블 데이터 분포도 확인.
print('레이블 테스트 세트 크기 :', y_test.shape)
print('테스트 세트 레이블 0 과 1의 분포도')
print(pd.Series(y_test).value_counts())

# Dummy Classifier로 학습/예측/정확도 평가
fakeclf = MyFakeClassifier()
fakeclf.fit(X_train , y_train)
fakepred = fakeclf.predict(X_test)
print('모든 예측을 0으로 하여도 정확도는:{:.3f}'.format(accuracy_score(y_test , fakepred)))
```

줄	코드	상세 설명
4	<code>pd.Series(y_test).value_counts()</code>	ndarray를 Series로 감싸 값 분포를 집계합니다. 결과는 0이 405건, 1이 45건으로 9:1의 심한 불균형입니다.
8~10	<code>fakeclf</code> 학습/예측	무조건 0만 반환하는 모델로 예측합니다.
10	<code>accuracy_score(y_test, fakepred)</code>	정확도가 0.900. 450건 중 0인 405건을 모두 맞혔기 때문입니다.

▶ 실행 결과

```
레이블 테스트 세트 크기 : (450,)
테스트 세트 레이블 0 과 1의 분포도
0    405
1     45
Name: count, dtype: int64
모든 예측을 0으로 하여도 정확도는:0.900
```

⚠ 결과 해석 — 정확도의 치명적 함정

아무것도 학습하지 않고 무조건 0만 찍은 모델이 90%의 정확도를 기록했습니다. 하지만 이 모델은 정작 찾아내야 할 7을 단 하나도 찾지 못했습니다.

암 진단, 사기 탐지, 불량품 검출처럼 찾아야 할 대상이 극히 드문 문제에서는 이런 일이 흔합니다. 암 환자가 1%인 데이터에서 "모두 정상"이라고 하면 정확도 99%지만 완전히 쓸모없는 모델입니다.

따라서 불균형 데이터에서는 정확도를 절대 단독으로 사용해서는 안 됩니다. 이것이 다음 절부터 배울 오차 행렬, 정밀도, 재현율이 필요한 이유입니다.

⚠ 버전 차이

pandas 2.x부터 `value_counts()` 결과 아래에 `Name: count, dtype: int64` 가 표시됩니다. 구버전에서는 `dtype: int64` 만 나왔습니다. 값 자체는 동일합니다.

2. 3.2 오차 행렬(Confusion Matrix)

2.1 오차 행렬의 구조

오차 행렬은 이진 분류에서 성능 지표로 잘 활용되며, 학습된 분류 모델이 예측을 수행하면서 얼마나 헷갈리고(confused) 있는지를 함께 보여 주는 지표입니다. 어떠한 유형의 예측 오류가 발생하는지를 나타냅니다.

	예측: Negative(0)	예측: Positive(1)
실제: Negative(0)	TN (True Negative) 0으로 예측했고 실제로도 0	FP (False Positive) 1로 예측했으나 실제로는 0
실제: Positive(1)	FN (False Negative) 0으로 예측했으나 실제로는 1	TP (True Positive) 1로 예측했고 실제로도 1

■ TP/FP/TN/FN 외우는 법

앞 글자 T/F는 예측이 맞았는지(True) 틀렸는지(False), 뒤 글자 P/N은 모델이 무엇으로 예측했는지를 의미합니다.

- **FP** = 예측이 틀렸고(F) 모델은 **Positive**로 예측 → 실제로는 Negative
- **FN** = 예측이 틀렸고(F) 모델은 **Negative**로 예측 → 실제로는 Positive

이 네 값으로 정확도, 정밀도, 재현율이 모두 계산됩니다.

$$\text{정확도} = (\text{TN} + \text{TP}) / (\text{TN} + \text{FP} + \text{FN} + \text{TP})$$

2.2 MyFakeClassifier의 오차 행렬

```
from sklearn.metrics import confusion_matrix

# 앞절의 예측 결과인 fakepred와 실제 결과인 y_test의 Confusion Matrix출력
confusion_matrix(y_test, fakepred)
```

줄	코드	상세 설명
4	<code>confusion_matrix(y_test, fakepred)</code>	첫 인자가 실제값 , 두 번째가 예측값 입니다. 순서를 바꾸면 행렬이 전치되어 완전히 다른 해석이 되므로 주의해야 합니다. 반환은 2×2 ndarray입니다.

▶ 실행 결과

```
array([[405,  0],
       [ 45,  0]])
```

■ 결과 해석 — 오차 행렬이 드러내는 진실

	예측 0	예측 1
실제 0	TN = 405	FP = 0
실제 1	FN = 45	TP = 0

정확도 90%라는 숫자 뒤에 숨겨져 있던 **진실이 드러납니다.**

- **TP가 0**입니다. 찾아야 할 7을 단 하나도 맞히지 못했습니다.
- **FN이 45**입니다. 실제 7인 45건을 전부 놓쳤습니다.
- 예측 1 열이 전부 0인 것은 모델이 **Positive 예측을 아예 하지 않았다**는 뜻입니다.

정확도라는 하나의 숫자로써 절대 알 수 없었던 정보를 오차 행렬이 한눈에 보여 줍니다.

⚠ 버전 차이

구버전(Windows 환경)에서는 `dtype=int64` 가 함께 표시되었으나, 현재 버전에서는 생략되어 출력됩니다. 값은 동일합니다.

3. 3.3 정밀도(Precision)와 재현율(Recall)

3.1 정의와 업무적 의미

정밀도와 재현율은 **Positive 데이터 세트의 예측 성능에 초점**을 맞춘 평가 지표입니다.

지표	공식	의미
정밀도 (Precision)	$TP / (FP + TP)$	Positive로 예측한 것 중 실제로 Positive인 비율. 양성 예측도라고도 합니다.
재현율 (Recall)	$TP / (FN + TP)$	실제 Positive인 것 중 Positive로 맞힌 비율. 민감도(Sensitivity) 또는 TPR 이라고도 합니다.

■ 어떤 지표가 더 중요한가 — 업무 특성에 따라 다르다

상황	중요 지표	이유
암 진단, 금융 사기 검출	재현율	실제 양성을 놓치면(FN) 생명이나 큰 손실 로 이어집니다. 오탐(FP)은 추가 검사로 해결 가능합니다.
스팸 메일 분류	정밀도	정상 메일을 스팸으로 분류하면(FP) 중요한 업무 메일을 놓칩니다. 스팸이 조금 들어오는 것(FN)은 견딜 만합니다.

재현율이 중요하면 **FN**을 낮추는 데, 정밀도가 중요하면 **FP**를 낮추는 데 초점을 맞춥니다.

3.2 MyFakeClassifier의 정밀도·재현율

```
from sklearn.metrics import accuracy_score, precision_score, recall_score
```



```
print("정밀도:", precision_score(y_test, fakepred))
print("재현율:", recall_score(y_test, fakepred))
```

줄	코드	상세 설명
3	<code>precision_score(y_test, fakepred)</code>	$TP / (FP + TP) = 0 / (0 + 0)$. 분모가 0 이라 계산이 불가능하지만, 사이킷런은 오류 대신 0 을 반환하고 경고를 출력합니다.
4	<code>recall_score(y_test, fakepred)</code>	$TP / (FN + TP) = 0 / (45 + 0) = 0.0$. 실제 7인 45건 중 하나도 못 맞췄다는 의미입니다.

▶ 실행 결과

```
정밀도: 0.0
재현율: 0.0
```

```
UndefinedMetricWarning: Precision is ill-defined and being set to 0.0 due to
no predicted samples. Use 'zero_division' parameter to control this behavior.
```

■ 결과 해석

정확도 90%였던 모델의 **정밀도와 재현율이 모두 0**입니다. 이 모델이 **완전히 무가치**하다는 사실이 명확히 드러납니다. 정확도만 봤다면 절대 알 수 없었을 것입니다.

경고 메시지의 `zero_division` 파라미터로 분모가 0일 때의 반환값을 제어할 수 있습니다. 예: `precision_score(y_test, fakepred, zero_division=0)` 으로 지정하면 경고 없이 0을 반환합니다.

3.3 평가 함수 생성

```
from sklearn.metrics import accuracy_score, precision_score, recall_score, confusion_matrix

def get_clf_eval(y_test, pred):
    confusion = confusion_matrix(y_test, pred)
    accuracy = accuracy_score(y_test, pred)
    precision = precision_score(y_test, pred)
    recall = recall_score(y_test, pred)
    print('오차 행렬')
    print(confusion)
    print('정확도: {0:.4f}, 정밀도: {1:.4f}, 재현율: {2:.4f}'.format(accuracy, precision, recall))
```

줄	코드	상세 설명
3	<code>def get_clf_eval(y_test, pred):</code>	세 지표와 오차 행렬을 한 번에 출력하는 함수 입니다. 앞으로 여러 임계값을 비교할 때 반복 호출되므로 함수화가 필수적입니다.
4~7	<code>confusion / accuracy / precision / recall</code>	네 가지 지표를 각각 계산합니다. 모두 (실제값, 예측값) 순서로 인자를 받습니다.
10	<code>print('정확도: {0:.4f}, ...'.format(...))</code>	세 지표를 한 줄에 소수점 4자리로 정리해 출력합니다.

■ 이 함수는 3장 전체에서 계속 확장됩니다

3.4장에서 **F1 스코어**가 추가되고, 3.5장에서 **ROC-AUC**가 추가되어 최종적으로 5개 지표를 출력하는 형태로 발전합니다. 완성된 버전은 3.6장과 4장 실습에서 계속 재사용됩니다.

3.4 타이타닉 로지스틱 회귀 평가

```
import numpy as np
import pandas as pd

from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression

# 원본 데이터를 재로딩, 데이터 가공, 학습데이터/테스트 데이터 분할.
titanic_df = pd.read_csv('./titanic_train.csv')
y_titanic_df = titanic_df['Survived']
X_titanic_df = titanic_df.drop('Survived', axis=1)
X_titanic_df = transform_features(X_titanic_df)

X_train, X_test, y_train, y_test = train_test_split(X_titanic_df, y_titanic_df, \
                                                    test_size=0.20, random_state=11)

lr_clf = LogisticRegression(solver='liblinear')

lr_clf.fit(X_train, y_train)
pred = lr_clf.predict(X_test)
get_clf_eval(y_test, pred)
```

줄	코드	상세 설명
13~14	<code>train_test_split(..., random_state=11)</code>	앞의 Dummy 예제(<code>random_state=0</code>)와 시드가 다릅니다 . 이후 3장 전체가 이 분할을 기준으로 진행됩니다.
16	<code>LogisticRegression(solver='liblinear')</code>	실제로 학습하는 진짜 모델 입니다. <code>liblinear</code> 는 작은 데이터에 적합한 최적화 알고리즘으로 수렴 경고를 방지합니다.
20	<code>get_clf_eval(y_test, pred)</code>	앞서 만든 함수로 세 지표를 한꺼번에 확인합니다.

▶ 실행 결과

```
오차 행렬
[[108  10]
 [ 14  47]]
정확도: 0.8659, 정밀도: 0.8246, 재현율: 0.7705
```

■ 결과 해석

- **TN=108**: 사망을 사망으로 정확히 예측 / **FP=10**: 사망인데 생존으로 오판
- **FN=14**: 생존인데 사망으로 오판 / **TP=47**: 생존을 생존으로 정확히 예측
- **정밀도 0.8246** = $47 / (10 + 47)$. 생존이라고 예측한 57명 중 47명이 실제 생존
- **재현율 0.7705** = $47 / (14 + 47)$. 실제 생존자 61명 중 47명을 찾아냄

정밀도가 재현율보다 높습니다. 즉 이 모델은 **생존 예측에는 신중하지만 실제 생존자를 일부 놓치는** 경향이 있습니다.

3.5 정밀도/재현율 트레이드오프

정밀도와 재현율은 **상호 보완적인 평가 지표**이기 때문에 어느 한쪽을 강제로 높이면 다른 하나는 떨어지기 쉽습니다. 이를 **정밀도/재현율의 트레이드오프(Trade-off)**라고 부릅니다.

사이킷런의 분류 알고리즘은 예측 데이터가 특정 레이블에 속하는지를 **예측 확률로 계산**합니다. 그리고 **일반적으로 임계값 (threshold) 0.5를 기준으로**, 확률이 이보다 크면 Positive, 작으면 Negative로 결정합니다. 이 **임계값을 조정하면 정밀도와 재현율의 균형을 바꿀 수** 있습니다.

3.6 predict_proba() 이해

```
pred_proba = lr_clf.predict_proba(X_test)
pred = lr_clf.predict(X_test)
print('pred_proba()결과 Shape : {0}'.format(pred_proba.shape))
print('pred_proba array에서 앞 3개만 샘플로 추출 \n:', pred_proba[:3])

# 예측 확률 array 와 예측 결과값 array 를 concatenate 하여 예측 확률과 결과값을 한눈에 확인
pred_proba_result = np.concatenate([pred_proba , pred.reshape(-1,1)],axis=1)
print('두개의 class 중에서 더 큰 확률을 클래스 값으로 예측 \n',pred_proba_result[:3])
```

줄	코드	상세 설명
1	pred_proba = lr_clf.predict_proba(X_test)	예측 확률 을 반환합니다. shape는 (179, 2) 로, 0번 열은 클래스 0일 확률, 1번 열은 클래스 1일 확률 입니다. 두 값의 합은 항상 1입니다.
2	pred = lr_clf.predict(X_test)	최종 예측 레이블 을 반환합니다.
7	np.concatenate([pred_proba, pred.reshape(-1,1)], axis=1)	확률 배열과 예측 배열을 옆으로 결합 합니다. pred 는 1차원이라 reshape(-1,1) 로 (179,1) 2차원 으로 바뀌어 결합됩니다. 결과는 (179, 3)입니다.

▶ 실행 결과

```
pred_proba()결과 Shape : (179, 2)
pred_proba array에서 앞 3개만 샘플로 추출
: [[0.44935227 0.55064773]
 [0.86335512 0.13664488]
 [0.86429645 0.13570355]]
두개의 class 중에서 더 큰 확률을 클래스 값으로 예측
[[0.44935227 0.55064773 1.
 [0.86335512 0.13664488 0.
 [0.86429645 0.13570355 0.
 ]
 ]
 ]]
```

■ 결과 해석 — predict()의 정체

첫 번째 샘플은 사망 확률 0.449, 생존 확률 0.551 → 더 큰 쪽인 **1(생존)**로 예측되었습니다. 즉 **predict()** 는 **predict_proba()** 의 결과에 **임계값 0.5를 적용한 것일** 뿐입니다.

주목할 점은 첫 샘플의 확률이 **0.551로 0.5에 매우 가깝다**는 것입니다. 임계값을 0.6으로만 올려도 이 샘플은 **0으로 바뀔**니다. 임계값 조정이 예측을 바꾸는 원리가 여기에 있습니다.

3.7 Binarizer 활용

```
from sklearn.preprocessing import Binarizer

X = [[ 1, -1, 2],
      [ 2, 0, 0],
      [ 0, 1.1, 1.2]]

# threshold 기준값보다 같거나 작으면 0을, 크면 1을 반환
```

```
binarizer = Binarizer(threshold=1.1)
print(binarizer.fit_transform(X))
```

줄	코드	상세 설명
7	<code>Binarizer(threshold=1.1)</code>	<code>Binarizer</code> 는 입력값을 기준값(threshold) 과 비교해 0 또는 1로 변환 하는 전처리 클래스입니다. 기준값보다 크면 1, 같거나 작으면 0 입니다. 1.1은 포함되지 않아 0 이 되는 점에 주의하세요.
8	<code>binarizer.fit_transform(X)</code>	변환을 수행합니다. 2와 1.2만 1.1보다 크므로 1이 되고 나머지는 모두 0이 됩니다.

▶ 실행 결과

```
[[0. 0. 1.]
 [1. 0. 0.]
 [0. 0. 1.]]
```

```
from sklearn.preprocessing import Binarizer

#Binarizer의 threshold 설정값. 분류 결정 임계값임.
custom_threshold = 0.5

# predict_proba( ) 반환값의 두번째 컬럼 , 즉 Positive 클래스 컬럼 하나만 추출하여 Binarizer를 적용
pred_proba_1 = pred_proba[:,1].reshape(-1,1)

binarizer = Binarizer(threshold=custom_threshold).fit(pred_proba_1)
custom_predict = binarizer.transform(pred_proba_1)

get_clf_eval(y_test, custom_predict)
```

줄	코드	상세 설명
7	<code>pred_proba_1 = pred_proba[:,1].reshape(-1,1)</code>	Positive(클래스 1) 확률 컬럼만 추출합니다. Binarizer는 2차원 입력만 받으므로 <code>reshape(-1,1)</code> 이 필수입니다.
9~10	<code>Binarizer(threshold=0.5).fit(...).transform(...)</code>	임계값 0.5로 변환합니다. 이는 <code>predict()</code> 와 완전히 동일한 동작입니다.
12	<code>get_clf_eval(y_test, custom_predict)</code>	결과가 앞의 <code>predict()</code> 평가와 정확히 일치합니다. 이로써 predict가 임계값 0.5의 Binarizer와 같음이 증명됩니다.

▶ 실행 결과

```
오차 행렬
[[108 10]
 [ 14 47]]
정확도: 0.8659, 정밀도: 0.8246, 재현율: 0.7705
```

```
# Binarizer의 threshold 설정값을 0.4로 설정. 즉 분류 결정 임계값을 0.5에서 0.4로 낮춤
custom_threshold = 0.4
pred_proba_1 = pred_proba[:,1].reshape(-1,1)
binarizer = Binarizer(threshold=custom_threshold).fit(pred_proba_1)
custom_predict = binarizer.transform(pred_proba_1)

get_clf_eval(y_test , custom_predict)
```

줄	코드	상세 설명
---	----	-------

2	custom_threshold = 0.4	임계값을 0.5에서 0.4로 낮춥니다 . 기준이 낮아졌으므로 더 많은 샘플이 Positive로 분류됩니다 .
---	------------------------	--

▶ 실행 결과

```
오차 행렬
[[97 21]
 [11 50]]
정확도: 0.8212, 정밀도: 0.7042, 재현율: 0.8197
```

■ 결과 해석 — 트레이드오프의 실체

임계값	FP	FN	정밀도	재현율
0.5	10	14	0.8246	0.7705
0.4	21	11	0.7042	0.8197

임계값을 낮추자 **재현율은 0.77 → 0.82로 올랐지만 정밀도는 0.82 → 0.70으로 떨어졌습니다**. Positive 판정을 남발하니 **FP가 10 → 21로 늘고 FN은 14 → 11로 줄어든** 것입니다. 이것이 트레이드오프의 실체입니다.

3.8 임계값 변화에 따른 지표 변화

```
# 테스트를 수행할 모든 임계값을 리스트 객체로 저장.
thresholds = [0.4, 0.45, 0.50, 0.55, 0.60]

def get_eval_by_threshold(y_test , pred_proba_c1, thresholds):
    # thresholds list객체내의 값을 차례로 iteration하면서 Evaluation 수행.
    for custom_threshold in thresholds:
        binarizer = Binarizer(threshold=custom_threshold).fit(pred_proba_c1)
        custom_predict = binarizer.transform(pred_proba_c1)
        print('임계값:', custom_threshold)
        get_clf_eval(y_test , custom_predict)

get_eval_by_threshold(y_test , pred_proba[:,1].reshape(-1,1), thresholds )
```

줄	코드	상세 설명
2	thresholds = [0.4, 0.45, 0.50, 0.55, 0.60]	테스트할 임계값 5개를 리스트로 준비합니다.
4	def get_eval_by_threshold(y_test, pred_proba_c1, thresholds):	여러 임계값을 반복 평가하는 함수 입니다. 임계값별 지표 변화를 한눈에 비교할 수 있습니다.
6~10	for custom_threshold in thresholds:	각 임계값마다 Binarizer를 새로 만들어 변환하고 평가합니다.

▶ 실행 결과

```
임계값: 0.4
오차 행렬
[[97 21]
 [11 50]]
정확도: 0.8212, 정밀도: 0.7042, 재현율: 0.8197
임계값: 0.45
오차 행렬
[[105 13]
 [ 13 48]]
정확도: 0.8547, 정밀도: 0.7869, 재현율: 0.7869
임계값: 0.5
```

```
오차 행렬
[[108  10]
 [ 14  47]]
정확도: 0.8659, 정밀도: 0.8246, 재현율: 0.7705
임계값: 0.55
오차 행렬
[[111   7]
 [ 16  45]]
정확도: 0.8715, 정밀도: 0.8654, 재현율: 0.7377
임계값: 0.6
오차 행렬
[[113   5]
 [ 17  44]]
정확도: 0.8771, 정밀도: 0.8980, 재현율: 0.7213
```

■ 결과 해석 — 명확한 반비례 관계

임계값	0.40	0.45	0.50	0.55	0.60
정밀도	0.7042	0.7869	0.8246	0.8654	0.8980 ↑
재현율	0.8197 ↑	0.7869	0.7705	0.7377	0.7213

임계값이 올라갈수록 정밀도는 꾸준히 상승하고 재현율은 꾸준히 하락합니다. 임계값 **0.45**에서 두 값이 **0.7869**로 일치하는 지점도 흥미롭습니다.

중요한 것은 "어떤 임계값이 정답인가"에 대한 절대적 기준은 없다는 점입니다. 업무 특성에 따라 결정해야 합니다.

3.9 precision_recall_curve()

```
from sklearn.metrics import precision_recall_curve

# 레이블 값이 1일때의 예측 확률을 추출
pred_proba_class1 = lr_clf.predict_proba(X_test)[: , 1]

# 실제값 데이터 셋과 레이블 값이 1일 때의 예측 확률을 precision_recall_curve 인자로 입력
precisions, recalls, thresholds = precision_recall_curve(y_test, pred_proba_class1)
print('반환된 분류 결정 임계값 배열의 Shape:', thresholds.shape)
print('반환된 precisions 배열의 Shape:', precisions.shape)
print('반환된 recalls 배열의 Shape:', recalls.shape)

print("thresholds 5 sample:", thresholds[:5])
print("precisions 5 sample:", precisions[:5])
print("recalls 5 sample:", recalls[:5])

#반환된 임계값 배열 로우가 147건이므로 샘플로 10건만 추출하되, 임계값을 15 Step으로 추출.
thr_index = np.arange(0, thresholds.shape[0], 15)
print('샘플 추출을 위한 임계값 배열의 index 10개:', thr_index)
print('샘플용 10개의 임계값: ', np.round(thresholds[thr_index], 2))

# 15 step 단위로 추출된 임계값에 따른 정밀도와 재현율 값
print('샘플 임계값별 정밀도: ', np.round(precisions[thr_index], 3))
print('샘플 임계값별 재현율: ', np.round(recalls[thr_index], 3))
```

출 코드

상세 설명

7	<code>precisions, recalls, thresholds = precision_recall_curve(y_test, pred_proba_class1)</code>	3개의 배열을 반환합니다. 일일이 임계값을 지정하지 않아도 가능한 모든 임계값 에서의 정밀도·재현율을 자동으로 계산해 줍니다.
8~10	<code>shape</code> 출력	중요한 특징: <code>precisions</code> 와 <code>recalls</code> 의 길이가 <code>thresholds</code> 보다 1 더 큼 니다. 마지막에 정밀도=1, 재현율=0인 경계점이 추가 되기 때문입니다.
17	<code>thr_index = np.arange(0, thresholds.shape[0], 15)</code>	임계값이 너무 많아 15개 간격으로 샘플링 합니다. <code>np.arange(시작, 끝, 간격)</code> 으로 인덱스 배열을 만듭니다.
19	<code>np.round(thresholds[thr_index], 2)</code>	팬시 인덱싱으로 해당 위치의 임계값만 추출해 소수점 2자리로 출력합니다.
22~23	<code>precisions[thr_index] / recalls[thr_index]</code>	같은 인덱스로 정밀도·재현율을 추출합니다. 세 배열의 인덱스가 서로 대응 하므로 가능한 방식입니다.

▶ 실행 결과

```
반환된 분류 결정 임계값 배열의 Shape: (165,)
반환된 precisions 배열의 Shape: (166,)
반환된 recalls 배열의 Shape: (166,)
thresholds 5 sample: [0.01974988 0.06956414 0.08402808 0.08474207 0.08920161]
precisions 5 sample: [0.34078212 0.34269663 0.34463277 0.34659091 0.34857143]
recalls 5 sample: [1. 1. 1. 1. 1.]
샘플 추출을 위한 임계값 배열의 index 10개: [ 0 15 30 45 60 75 90 105 120 135 150]
샘플용 10개의 임계값: [0.02 0.11 0.13 0.14 0.16 0.24 0.32 0.45 0.62 0.73 0.87]
샘플 임계값별 정밀도: [0.341 0.372 0.401 0.44 0.505 0.598 0.688 0.774 0.915 0.968 0.938]
샘플 임계값별 재현율: [1. 1. 0.967 0.902 0.902 0.902 0.869 0.787 0.705 0.492 0.246]
```

■ 결과 해석

임계값이 **0.02일 때 재현율 1.0**(모두 Positive로 예측하니 다 찾아냄)이지만 **정밀도는 0.341**에 불과합니다. 반대로 임계값 **0.87에서는 정밀도 0.938**로 매우 높지만 **재현율은 0.246**으로 폭락합니다. 트레이드오프가 극단적으로 드러납니다.

⚠ 버전 차이 — 반환 배열 크기

교재(사이킷런 1.0.2)에서는 `thresholds`가 **147개**였으나 현재 버전(1.7.2)에서는 **165개**가 반환됩니다. 사이킷런이 **중복 임계값 처리 방식을 개선**하면서 생긴 차이입니다. 곡선의 형태와 결론은 동일하지만, 주석의 "147건"이나 인덱스 개수는 버전에 따라 달라질 수 있습니다.

3.10 정밀도-재현율 곡선 시각화

```
import matplotlib.pyplot as plt
import matplotlib.ticker as ticker
%matplotlib inline

def precision_recall_curve_plot(y_test, pred_proba_c1):
    # threshold ndarray와 이 threshold에 따른 정밀도, 재현율 ndarray 추출.
    precisions, recalls, thresholds = precision_recall_curve(y_test, pred_proba_c1)

    # X축을 threshold값으로, Y축은 정밀도, 재현율 값으로 각각 Plot 수행. 정밀도는 점선으로 표시
    plt.figure(figsize=(8,6))
    threshold_boundary = thresholds.shape[0]
    plt.plot(thresholds, precisions[0:threshold_boundary], linestyle='--', label='precision')
    plt.plot(thresholds, recalls[0:threshold_boundary], label='recall')

    # threshold 값 X 축의 Scale을 0.1 단위로 변경
```

```

start, end = plt.xlim()
plt.xticks(np.round(np.arange(start, end, 0.1),2))

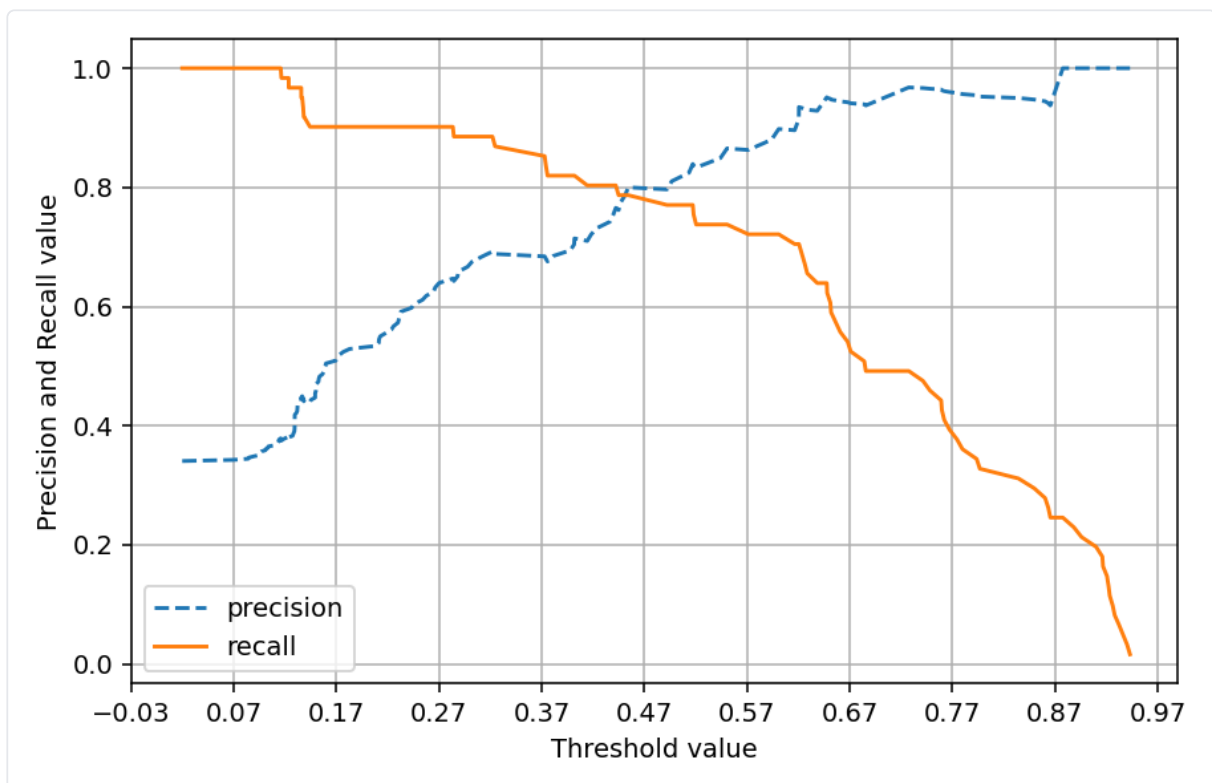
# x축, y축 label과 legend, 그리고 grid 설정
plt.xlabel('Threshold value'); plt.ylabel('Precision and Recall value')
plt.legend(); plt.grid()
plt.show()

```

```
precision_recall_curve_plot( y_test, lr_clf.predict_proba(X_test)[: , 1] )
```

줄	코드	상세 설명
11	threshold_boundary = thresholds.shape[0]	핵심 처리입니다. precisions/recalls가 thresholds보다 1 길기 때문에, 길이를 맞추지 않으면 plot에서 오류가 납니다. 이 변수로 개수를 저장합니다.
12	plt.plot(thresholds, precisions[0:threshold_boundary], linestyle='--', label='precision')	정밀도를 점선(--) 으로 그립니다. [0:threshold_boundary] 로 마지막 원소를 잘라내 길이를 맞춥니다.
13	plt.plot(thresholds, recalls[0:threshold_boundary], label='recall')	재현율을 실선 으로 그립니다.
16~17	start, end = plt.xlim() / plt.xticks(...)	현재 x축 범위를 얻어 0.1 간격의 눈금 으로 다시 설정합니다. 임계값을 읽기 쉬워집니다.
20~21	xlabel / ylabel / legend / grid	축 이름, 범례, 격자를 설정해 그래프를 완성합니다.

▶ 실행 결과



임계값에 따른 정밀도(점선)와 재현율(실선)의 변화

■ 그래프 읽는 법

- 임계값이 커질수록 **정밀도(점선)**는 상승, **재현율(실선)**은 하락하는 뚜렷한 반비례 곡선입니다.
- 두 곡선이 **교차하는 지점**이 정밀도와 재현율이 균형을 이루는 임계값입니다(약 0.45).
- 임계값이 **0.8을 넘어가면 정밀도가 오히려 흔들리는데**, 이는 Positive 예측 건수가 너무 적어져 **표본이 불안정**해지기 때문입니다.
- 업무 요구사항이 "재현율 80% 이상"이라면, 그래프에서 재현율 0.8에 해당하는 임계값(약 0.4)을 선택하면 됩니다.

⚠ 정밀도·재현율만 조정하면 안 된다

임계값을 극단적으로 조정하면 한 지표를 100%로 만들 수 있습니다. 모두 **Positive로 예측하면 재현율 100%, 확실한 것 하나만 Positive로 예측하면 정밀도 100%**입니다. 하지만 둘 다 무의미한 모델입니다. 따라서 두 지표는 반드시 **함께 봐야** 하며, 이를 하나의 숫자로 결합한 것이 다음 절의 **F1 스코어**입니다.

4.3.4 F1 스코어

F1 스코어는 정밀도와 재현율을 결합한 지표입니다. 정밀도와 재현율이 어느 한쪽으로 치우치지 않는 수치를 나타낼 때 **상대적으로 높은 값**을 가집니다.

$$F1 = 2 \times (\text{precision} \times \text{recall}) / (\text{precision} + \text{recall})$$

■ 왜 산술 평균이 아니라 조화 평균인가

정밀도 0.9, 재현율 0.1인 극단적인 모델을 보겠습니다.

- **산술 평균**: $(0.9 + 0.1) / 2 = 0.5$ — 그럭저럭 괜찮아 보입니다.
- **조화 평균(F1)**: $2 \times (0.9 \times 0.1) / (0.9 + 0.1) = 0.18$ — 형편없다는 것이 드러납니다.

조화 평균은 **작은 값에 훨씬 큰 영향**을 받으므로, **한쪽으로 치우친 모델에 낮은 점수**를 줍니다. 이것이 F1 스코어가 균형을 평가하는 원리입니다.

```
from sklearn.metrics import f1_score
f1 = f1_score(y_test, pred)
print('F1 스코어: {0:.4f}'.format(f1))
```

줄	코드	상세 설명
2	f1_score(y_test, pred)	다른 지표와 동일하게 (실제값, 예측값)을 받습니다. 임계값 0.5 기준 정밀도 0.8246, 재현율 0.7705로 계산하면 0.7966이 나옵니다.

▶ 실행 결과

F1 스코어: 0.7966

```
def get_clf_eval(y_test, pred):
    confusion = confusion_matrix(y_test, pred)
    accuracy = accuracy_score(y_test, pred)
    precision = precision_score(y_test, pred)
    recall = recall_score(y_test, pred)
    # F1 스코어 추가
```

```

f1 = f1_score(y_test, pred)
print('오차 행렬')
print(confusion)
# f1 score print 추가
print('정확도: {0:.4f}, 정밀도: {1:.4f}, 재현율: {2:.4f}, F1:{3:.4f}'.format(accuracy, precision, recall, f1))

thresholds = [0.4, 0.45, 0.50, 0.55, 0.60]
pred_proba = lr_clf.predict_proba(X_test)
get_eval_by_threshold(y_test, pred_proba[:,1].reshape(-1,1), thresholds)

```

줄	코드	상세 설명
1~11	def get_clf_eval(y_test, pred): ... f1 추가	기존 함수에 F1 스코어 를 추가해 재정의합니다. 파이썬은 같은 이름의 함수를 다시 정의하면 덮어쓰므로 , 이후 <code>get_eval_by_threshold</code> 가 호출하는 것도 자동으로 새 함수가 됩니다.
15	get_eval_by_threshold(...)	함수를 수정하지 않았는데도 F1이 포함된 결과 가 출력됩니다.

▶ 실행 결과

```

임계값: 0.4
오차 행렬
[[97 21]
 [11 50]]
정확도: 0.8212, 정밀도: 0.7042, 재현율: 0.8197, F1:0.7576
임계값: 0.45
오차 행렬
[[105 13]
 [ 13 48]]
정확도: 0.8547, 정밀도: 0.7869, 재현율: 0.7869, F1:0.7869
임계값: 0.5
오차 행렬
[[108 10]
 [ 14 47]]
정확도: 0.8659, 정밀도: 0.8246, 재현율: 0.7705, F1:0.7966
임계값: 0.55
오차 행렬
[[111 7]
 [ 16 45]]
정확도: 0.8715, 정밀도: 0.8654, 재현율: 0.7377, F1:0.7965
임계값: 0.6
오차 행렬
[[113 5]
 [ 17 44]]
정확도: 0.8771, 정밀도: 0.8980, 재현율: 0.7213, F1:0.8000

```

■ 결과 해석

임계값	0.40	0.45	0.50	0.55	0.60
정밀도	0.7042	0.7869	0.8246	0.8654	0.8980
재현율	0.8197	0.7869	0.7705	0.7377	0.7213
F1	0.7576	0.7869	0.7966	0.7965	0.8000

정밀도와 재현율이 급격히 변하는 동안 **F1은 0.75~0.80 사이로 완만하게 움직입니다**. 두 지표를 결합했기 때문입니다. 이 데이터에서는 **임계값 0.6에서 F1이 0.8000으로 최대가 됩니다**.

다만 F1이 높다고 항상 좋은 모델은 아닙니다. **재현율이 결정적으로 중요한 업무**라면 F1이 조금 낮더라도 임계값 0.4를 선택하는 것이 좋습니다. **지표는 판단의 근거일 뿐 결정은 업무 맥락에서 내려야 합니다**.

5.3.5 ROC 곡선과 AUC

5.1 ROC 곡선의 개념

ROC 곡선(Receiver Operating Characteristic Curve)은 원래 **2차 세계대전 때 통신 장비 성능 평가**를 위해 개발된 수신자 판단 곡선입니다. 의학 분야에서 많이 사용되며, 머신러닝의 이진 분류 성능 평가에도 중요하게 쓰입니다.

ROC 곡선은 **FPR(False Positive Rate)이 변할 때 TPR(True Positive Rate)이 어떻게 변하는지**를 나타내는 곡선입니다.

지표	공식	의미
TPR (민감도, 재현율)	$TP / (FN + TP)$	실제 Positive 를 정확히 예측한 비율
TNR (특이성, Specificity)	$TN / (FP + TN)$	실제 Negative 를 정확히 예측한 비율
FPR	$FP / (FP + TN) = 1 - TNR$	실제 Negative 를 잘못 Positive 로 예측한 비율

■ 임계값과 FPR의 관계

- 임계값을 **1로 하면** Positive 예측을 아예 하지 않으므로 **FP=0, FPR=0**이 됩니다.
- 임계값을 **0으로 하면** 모두 Positive로 예측하므로 **TN=0, FPR=1**이 됩니다.

즉 **임계값을 1에서 0으로 낮춰 가며 FPR과 TPR의 변화를 추적한 것이 ROC 곡선**입니다.

5.2 roc_curve() 활용

```
from sklearn.metrics import roc_curve

# 레이블 값이 1일때의 예측 확률을 추출
pred_proba_class1 = lr_clf.predict_proba(X_test)[: , 1]

fprs , tprs , thresholds = roc_curve(y_test, pred_proba_class1)
# 반환된 임계값 배열에서 샘플로 데이터를 추출하되, 임계값을 5 Step으로 추출.
# thresholds[0]은 max(예측확률)+1로 임의 설정됨. 이를 제외하기 위해 np.arange는 1부터 시작
thr_index = np.arange(1, thresholds.shape[0], 5)
print('샘플 추출을 위한 임계값 배열의 index:', thr_index)
```

```
print('샘플 index로 추출한 임계값: ', np.round(thresholds[thr_index], 2))

# 5 step 단위로 추출된 임계값에 따른 FPR, TPR 값
print('샘플 임계값별 FPR: ', np.round(fprs[thr_index], 3))
print('샘플 임계값별 TPR: ', np.round(tprs[thr_index], 3))
```

줄	코드	상세 설명
6	fprs, tprs, thresholds = roc_curve(y_test, pred_proba_class1)	<code>precision_recall_curve</code> 와 유사하게 3개 배열을 반환 합니다. 인자도 (실제 값, 예측 확률)로 동일합니다.
9	thr_index = np.arange(1, thresholds.shape[0], 5)	0 이 아니라 1 부터 시작하는 것이 핵심입니다. 주석대로 <code>thresholds[0]</code> 은 사이킷런이 곡선의 시작점을 만들기 위해 max(예측확률)+1로 임의 설정한 값 이라 의미가 없으므로 제외합니다.
13~14	fprs[thr_index] / tprs[thr_index]	동일한 인덱스로 FPR과 TPR을 추출합니다.

▶ 실행 결과

```
샘플 추출을 위한 임계값 배열의 index: [ 1  6 11 16 21 26 31 36 41 46]
샘플 index로 추출한 임계값: [0.94 0.73 0.62 0.52 0.44 0.28 0.15 0.14 0.13 0.12]
샘플 임계값별 FPR: [0.    0.008 0.025 0.076 0.127 0.254 0.576 0.61  0.746 0.847]
샘플 임계값별 TPR: [0.016 0.492 0.705 0.738 0.803 0.885 0.902 0.951 0.967 1.   ]
```

■ 결과 해석

임계값이 **0.94에서 0.12로 낮아짐**에 따라 **FPR은 0 → 0.847로, TPR은 0.016 → 1.0으로** 함께 증가합니다. 주목할 점은 **TPR이 FPR보다 훨씬 빠르게 상승**한다는 것입니다. 예를 들어 임계값 0.73에서 **FPR은 0.008에 불과한데 TPR은 이미 0.492**입니다. 이는 모델이 **Positive와 Negative를 잘 구분**하고 있다는 좋은 신호입니다.

5.3 ROC 곡선 시각화

```
def roc_curve_plot(y_test , pred_proba_c1):
    # 임계값에 따른 FPR, TPR 값을 반환 받음.
    fprs , tprs , thresholds = roc_curve(y_test ,pred_proba_c1)

    # ROC Curve를 plot 곡선으로 그림.
    plt.plot(fprs , tprs, label='ROC')
    # 가운데 대각선 직선을 그림.
    plt.plot([0, 1], [0, 1], 'k--', label='Random')

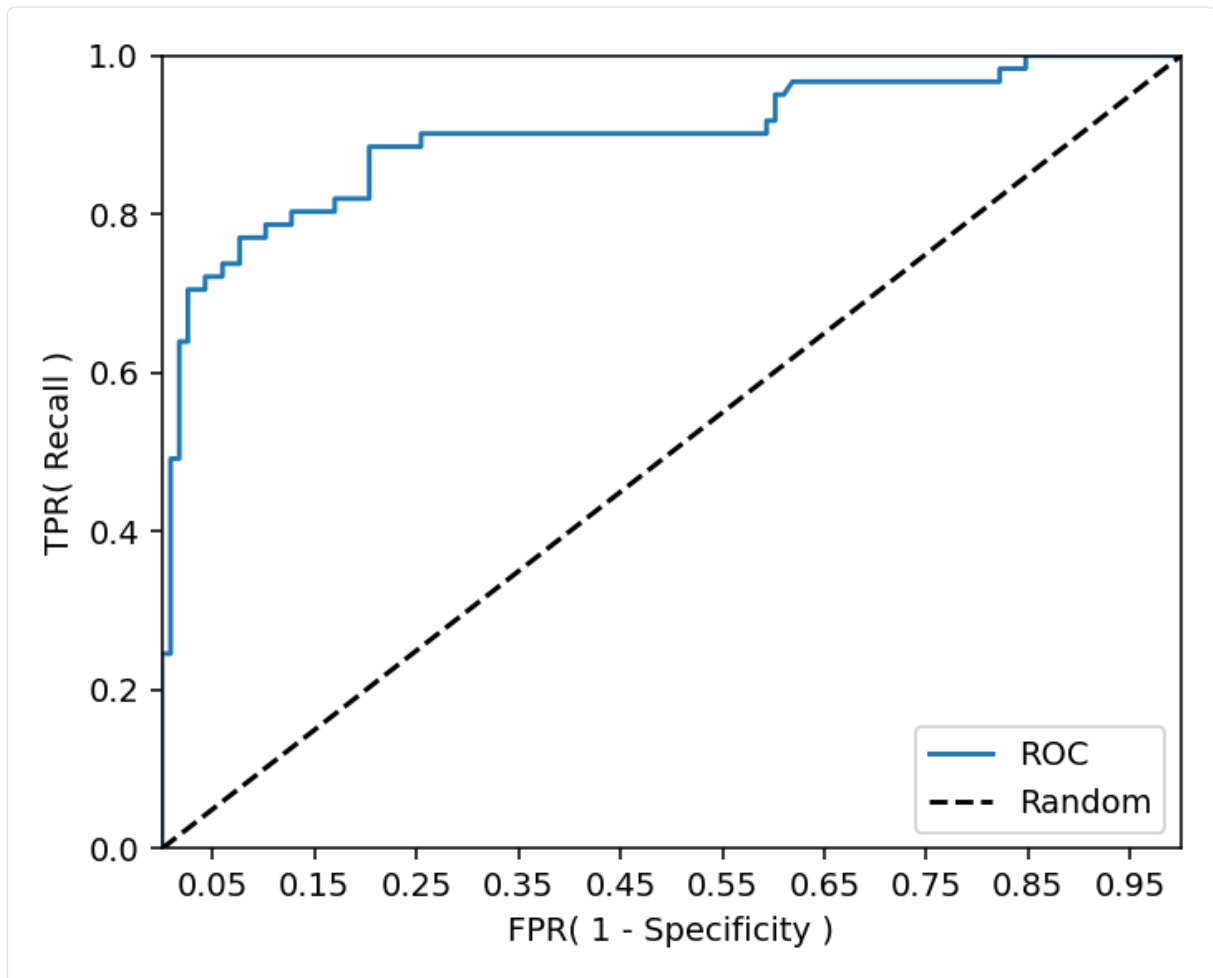
    # FPR X 축의 Scale을 0.1 단위로 변경, X,Y 축명 설정등
    start, end = plt.xlim()
    plt.xticks(np.round(np.arange(start, end, 0.1),2))
    plt.xlim(0,1); plt.ylim(0,1)
    plt.xlabel('FPR( 1 - Specificity )'); plt.ylabel('TPR( Recall )')
    plt.legend()
    plt.show()

roc_curve_plot(y_test, lr_clf.predict_proba(X_test)[: , 1] )
```

줄	코드	상세 설명
6	plt.plot(fprs, tprs, label='ROC')	x축 FPR, y축 TPR 로 곡선을 그림니다. 정밀도-재현율 곡선과 달리 x축이 임계값이 아니라 FPR 입니다.

8	<code>plt.plot([0, 1], [0, 1], 'k--', label='Random')</code>	(0,0)에서 (1,1)로 가는 검은 점선을 그립니다. 'k--'는 k=검정, --=점선입니다. 이 선은 랜덤 추측 수준의 성능을 의미하는 기준선입니다.
13	<code>plt.xlim(0,1); plt.ylim(0,1)</code>	두 축의 범위를 0~1로 고정합니다. ROC 곡선은 항상 이 정사각형 안에 그려집니다.
14	<code>plt.xlabel('FPR(1 - Specificity)')</code>	축 이름에 FPR = 1 - 특이성 이라는 관계를 함께 표기해 이해를 돕습니다.

▶ 실행 결과



ROC 곡선 — 실선이 모델, 점선이 랜덤 기준선

■ 그래프 읽는 법

- 곡선이 가운데 점선(랜덤)에서 멀리 떨어져 왼쪽 위로 붙을수록 좋은 모델입니다.
- 왼쪽 위 모서리 (0, 1)이 이상적인 지점입니다. FPR은 0이면서 TPR은 1, 즉 오탐 없이 모두 찾아내는 완벽한 분류입니다.
- 점선과 일치하면 랜덤 추측과 다를 바 없는 모델이고, 점선 아래로 내려가면 랜덤보다도 못한 것입니다.
- 본 그래프의 곡선은 초반에 가파르게 상승한 뒤 완만해집니다. FPR을 조금만 허용해도 TPR이 크게 오르는 좋은 모델의 형태입니다.

5.4 roc_auc_score()

ROC 곡선은 FPR과 TPR의 변화 양상을 보는 데 이용하며, 분류 성능 지표로 사용되는 것은 곡선 면적에 기반한 AUC 값입니다.

```
from sklearn.metrics import roc_auc_score
pred_proba = lr_clf.predict_proba(X_test)[: , 1]
```

```
roc_score = roc_auc_score(y_test, pred_proba)
print('ROC AUC 값: {:.4f}'.format(roc_score))
```

줄	코드	상세 설명
2	<code>pred_proba = lr_clf.predict_proba(X_test)[: , 1]</code>	예측 확률 을 준비합니다.
3	<code>roc_auc_score(y_test, pred_proba)</code>	매우 중요한 차이점 입니다. 정확도·정밀도·재현율·F1은 모두 예측 레이블 을 받지만, ROC-AUC 만은 예측 확률 을 받습니다. 레이블을 넣으면 값이 잘못 계산되므로 반드시 주의해야 합니다.

▶ 실행 결과

ROC AUC 값: 0.8987

■ AUC 값 해석 기준

AUC	평가
1.0	완벽한 분류
0.9 ~ 1.0	매우 우수
0.8 ~ 0.9	우수 (본 모델 0.8987)
0.7 ~ 0.8	보통
0.5	랜덤 추측 수준 — 무의미
0.5 미만	랜덤보다 나쁨(예측을 뒤집으면 오히려 나아짐)

AUC 0.8987은 거의 **0.9**에 근접한 **우수한 성능**입니다.

■ AUC의 결정적 장점 — 임계값에 무관하다

정확도·정밀도·재현율·F1은 모두 특정 **임계값**(보통 **0.5**)에서 계산된 값이라, 임계값이 바뀌면 값도 바뀝니다. 반면 **AUC**는 모든 **임계값**을 종합한 면적이므로 임계값 선택과 무관하게 모델의 근본적인 판별 능력을 측정합니다.

또한 **불균형 데이터**에서도 **정확도**처럼 왜곡되지 않습니다. 앞의 MyFakeClassifier처럼 모두 0으로 예측하는 모델은 정확도 90%였지만 **AUC**는 **0.5**가 나옵니다. 이것이 실무에서 이진 분류 성능 평가에 AUC를 널리 쓰는 이유입니다.

5.5 최종 통합 평가 함수

```
def get_clf_eval(y_test, pred=None, pred_proba=None):
    confusion = confusion_matrix(y_test, pred)
    accuracy = accuracy_score(y_test, pred)
    precision = precision_score(y_test, pred)
    recall = recall_score(y_test, pred)
    f1 = f1_score(y_test, pred)
    # ROC-AUC 추가
    roc_auc = roc_auc_score(y_test, pred_proba)
    print('오차 행렬')
    print(confusion)
    # ROC-AUC print 추가
```

```
print('정확도: {0:.4f}, 정밀도: {1:.4f}, 재현율: {2:.4f},\n
      F1: {3:.4f}, AUC:{4:.4f}'.format(accuracy, precision, recall, f1, roc_auc))
```

줄	코드	상세 설명
1	def get_clf_eval(y_test, pred=None, pred_proba=None):	인자가 3개로 늘어난 최종 버전입니다. <code>pred_proba</code> 가 추가된 이유는 ROC-AUC만 확률이 필요하기 때문입니다. 기본값 <code>None</code> 을 주어 인자 없이도 호출은 가능하게 했습니다.
2~6	confusion / accuracy / precision / recall / f1	다섯 지표는 예측 레이블(pred) 로 계산합니다.
8	roc_auc = roc_auc_score(y_test, pred_proba)	ROC-AUC만 예측 확률(pred_proba) 로 계산합니다.
12~13	print('정확도: ... AUC:{4:.4f}')	5개 지표를 한 줄로 출력합니다. <code>\</code> 는 문자열을 다음 줄로 이어 쓰기 위한 연결 문자입니다.

■ 이 함수가 3장의 최종 산출물입니다

3.1장의 정확도 하나에서 시작해, 오차 행렬 → 정밀도·재현율 → F1 → ROC-AUC까지 단계적으로 확장한 결과입니다. 이 함수는 3.6장(피마 인디언 당뇨병)과 4장 전체 실습에서 그대로 재사용되므로 반드시 이해하고 넘어가야 합니다.

6. 핵심 요약 및 머신러닝 활용 포인트

6.1 5가지 평가 지표 총정리

지표	공식	입력	특징 / 주의
정확도 Accuracy	$(TN+TP) / \text{전체}$	레이블	직관적이지만 불균형 데이터에서 왜곡
정밀도 Precision	$TP / (FP+TP)$	레이블	Positive 예측의 정확성. 스팸 분류 에서 중요
재현율 Recall	$TP / (FN+TP)$	레이블	실제 Positive 검출률. 암 진단·사기 탐지 에서 중요
F1	$2PR / (P+R)$	레이블	정밀도·재현율의 조화 평균 . 균형 평가
ROC-AUC	ROC 곡선 아래 면적	확률	임계값 무관 , 불균형에 강함. 0.5=랜덤

6.2 핵심 API 요약표

API	역할	주의 사항
<code>accuracy_score(y, pred)</code>	정확도	모두 (실제값, 예측 레이블) 순서
<code>precision_score(y, pred)</code>	정밀도	
<code>recall_score(y, pred)</code>	재현율	
<code>f1_score(y, pred)</code>	F1 스코어	
<code>confusion_matrix(y, pred)</code>	오차 행렬	반환은 <code>[[TN, FP], [FN, TP]]</code>
<code>roc_auc_score(y, proba)</code>	ROC-AUC	예측 확률 을 인자로 받음

<code>precision_recall_curve(y, proba)</code>	임계값별 정밀도·재현율	precisions/recalls가 thresholds보다 1 킵
<code>roc_curve(y, proba)</code>	임계값별 FPR·TPR	<code>thresholds[0]</code> 은 의미 없는 임의값
<code>Binarizer(threshold=t)</code>	임계값 기준 0/1 변환	2차원 입력 필요, 기준값 초과만 1
<code>predict_proba(X)</code>	클래스별 예측 확률	반환 (n, 2), Positive는 <code>[:, 1]</code>

6.3 머신러닝 활용 포인트

- **정확도만으로 판단하지 않기** : 특히 불균형 데이터에서는 반드시 오차 행렬과 정밀도·재현율을 함께 확인하세요.
- **업무 특성이 지표를 결정한다** : 무엇을 놓치면 안 되는지(재현율) 무엇을 잘못 잡으면 안 되는지(정밀도)를 먼저 정의하고 지표를 고르세요.
- **임계값은 조정 가능한 설계 변수** : 0.5는 기본값일 뿐입니다. 정밀도·재현율 곡선을 보고 업무 목표에 맞는 임계값을 선택하세요.
- **ROC-AUC는 모델 비교의 기본** : 임계값과 무관하므로 **여러 모델의 근본적인 성능을 비교할 때** 가장 적합합니다.
- **확률 vs 레이블 구분** : ROC-AUC에 레이블을 넣는 실수가 매우 흔합니다. **AUC만 확률**이라고 외워 두세요.
- **평가 함수를 만들어 재사용** : 본 장의 `get_clf_eval()` 처럼 지표를 묶어 두면 여러 모델·임계값 비교가 훨씬 수월해집니다.

⚠ 안정화 버전 참고 (scikit-learn 1.7.x 기준)

- `precision_recall_curve()` 의 **반환 배열 크기가 구버전과 다릅니다**(교재 147 → 현재 165). 중복 임계값 처리 방식이 개선된 결과이며, 곡선 형태와 결론은 동일합니다.
- 정밀도·재현율 계산 시 분모가 0이면 `UndefinedMetricWarning` 이 발생합니다. `zero_division=0` 파라미터로 제어할 수 있습니다.
- `Binarizer` 는 **기준값 초과일 때만 1**입니다(같으면 0). 경계값 처리에 주의하세요.
- pandas 2.x부터 `value_counts()` 출력에 `Name: count` 가 함께 표시됩니다.
- 다중 분류에서는 `average` 파라미터(`'macro'` , `'micro'` , `'weighted'`)를 지정해야 합니다. 본 장은 모두 이진 분류라 기본값 `'binary'` 가 적용되었습니다.